

University of Ottawa

CSI 3140 - WWW Structures, Techniques and Standards

Laboratory 5

Moodle-Based Integrated Web Application

Local Deployment, Secure Plugin Development, Authentication, Authorization, and JSON Data Exchange

Professor	Mohamed Ali Ibrahim, PhD, PEng
Term	Spring/Summer 2026
Lab Period	Monday, July 13, 2026 to Friday, July 24, 2026
Submission	Brightspace, by Friday, July 24, 2026, 11:59 PM
Team Size	2-3 students
Core Platform	Moodle LMS installed locally in a development environment
Technologies	Moodle, PHP, MariaDB/MySQL, HTML5, CSS3, JavaScript, Fetch/AJAX, JSON, Moodle Access API, Moodle Database API, browser developer tools, and an API testing tool

1. Laboratory Overview

The purpose of this laboratory is to integrate the major client-side, server-side, API, authentication, authorization, and security concepts studied in CSI 3140 within a real extensible Web platform. Each team will deploy Moodle locally and develop a small secure Moodle local plugin called Secure Course Hub.

Unlike a project that builds authentication from the beginning, this laboratory requires students to integrate with Moodle's existing authentication and session mechanisms. Students must identify the authenticated user, enforce role- and capability-based authorization, apply ownership checks, process server-side data, and exchange JSON data with JavaScript.

The laboratory continues the client-server and REST/JSON work of Laboratory 4, but moves the implementation into a PHP-based learning management system. The emphasis is not on changing Moodle core or reproducing the entire LMS. The emphasis is on extending a mature application safely through a plugin and understanding how platform APIs act as security and integration boundaries.

This is a programming laboratory. Each team must design, write, test, and demonstrate original PHP, JavaScript, XML, and Moodle plugin code. Installing and configuring Moodle alone does not satisfy the laboratory requirements.

2. Topics Covered in Lab 5

Topic Area	Required Coverage
Local Moodle deployment	Web server, PHP runtime, database, Moodle installation, configuration, development mode, and local testing.
Moodle plugin architecture	Local plugin structure, version metadata, language strings, access rules, page setup, and separation from Moodle core.
Authentication and sessions	Moodle login, session cookie, authenticated user context, require_login(), logout, and session expiry.
Authorization and RBAC	Roles, capabilities, contexts, require_capability(), 401/403 concepts, and server-side access decisions.
Ownership checks	Restricting students to their own records and allowing teachers to manage only records in authorized courses.

Server-side development	PHP request handling, Moodle page lifecycle, input processing, business rules, database operations, and safe responses.
JSON and AJAX	JavaScript fetch(), JSON requests and responses, dynamic page updates, and error handling.
Security controls	CSRF protection with sesskey, XSS prevention, input validation, output escaping, safe errors, and no secrets in source code.
Privacy and data minimization	Collecting only required data, avoiding exposure of student information, and documenting retention and access rules.
Testing and debugging	Role-based tests, negative security tests, browser developer tools, PHP/Moodle debugging, and evidence-based verification.

3. Learning Objectives

- Install and configure a local Moodle development instance.
- Explain Moodle's browser-server-database architecture and plugin extension model.
- Create a small local Moodle plugin without modifying Moodle core files.
- Use Moodle's existing login and session mechanisms to identify the current user.
- Distinguish authentication from authorization in a real application.
- Define and enforce Moodle capabilities for students, teachers, and administrators.
- Apply ownership and course-context checks in addition to role checks.
- Process form or JSON input on the server and validate all untrusted data.
- Use the Moodle Database API to create, read, update, and delete project records.
- Use JavaScript fetch() to exchange JSON data with a secure server-side endpoint.
- Apply CSRF, XSS, session, privacy, and error-handling protections.
- Test successful and denied access scenarios and explain the resulting behavior.
- Prepare a technical report describing architecture, security decisions, testing, and limitations.

4. Required Tasks and Acceptance Criteria

Complete all twelve tasks below. A task is complete only when the required files exist, the stated behaviour works with the specified test accounts, and the listed evidence is included in the submission. Equivalent implementations require prior approval.

Task and required implementation	Required files / configuration	Acceptance criterion and evidence
1. Install Moodle locally Run Moodle locally with a Web server, PHP, and MariaDB/MySQL. Record the exact Moodle and PHP versions.	Local Moodle installation; README.md environment section.	The Moodle home page and administration page open without fatal errors. Include one screenshot of the running site and the recorded versions.
2. Create test users and course Create exactly four test accounts: one administrator, one teacher, Student A, and Student B. Create one course and enrol the teacher and both students with the correct roles.	Moodle accounts, course, and enrolments.	The Participants page shows the teacher and both students with the expected roles. Use synthetic names and data only.
3. Create and install the plugin skeleton Create local_securecoursehub under local/securecoursehub. Install it through Site administration without modifying Moodle core.	version.php; language file; index.php; README.md.	Moodle reports the plugin as installed, and the plugin page loads for an authenticated authorized user.
4. Create the database table Define one request table with id, courseid, userid, title, description, status, response, timecreated, and timemodified. Allowed statuses: open, inprogress, resolved.	db/install.xml.	The table is created during installation. A new request stores the authenticated user id, course id, status open, and both timestamps.
5. Define capabilities Define viewown, createrequest, and managecourserequests. Students may view/create their own requests; editing teachers may manage course requests.	db/access.php and language strings.	Student A can create/view own requests but cannot manage the course queue. The teacher can manage requests in the demonstration course.

Task and required implementation	Required files / configuration	Acceptance criterion and evidence
6. Protect every server entry point Load config.php, require authentication, establish the course context, and enforce the required capability before reading or changing data.	index.php; ajax.php or approved service; relevant classes.	Unauthenticated access is redirected or rejected before data is returned. A logged-in user without permission receives access denied.
7. Implement student operations Allow an enrolled student to create a request, list only their own requests, edit their own open request, and delete their own open request. Resolved requests cannot be edited or deleted by a student.	Page/controller code; request service; form or JSON handling.	Student A completes all allowed operations. Student A cannot view, edit, or delete Student B's request by changing an id or URL parameter.
8. Implement teacher operations Allow the teacher to list course requests, filter by status, set status to open/inprogress/resolved, and add or update a response of at most 500 characters.	Teacher interface; request service; endpoint handling.	The teacher manages requests in the authorized course and is denied access in a course where the capability is not granted.
9. Use the Moodle Database API Implement CRUD with \$DB methods. Validate existence, course association, ownership, and permitted state transitions before each write.	classes/local/request_service.php or equivalent.	No SQL concatenates user input. Missing records return a safe not-found response. Failed validation leaves the database unchanged.
10. Implement JSON and fetch() Implement at least one state-changing operation using fetch() and JSON. Update the page without a full reload. Recommended: teacher status update.	ajax.php or Moodle external service; JavaScript module/file.	Browser Network tools show a JSON request and response. The page updates after success and shows a visible message after failure.
11. Apply required security controls Validate sesskey for every state-changing action; validate type, length, and allowed values; escape untrusted output; do not expose secrets or internal errors.	All PHP endpoints and rendering code.	Invalid sesskey is rejected. Script markup is displayed as text. Invalid status and overlength input are rejected.
12. Test, document, and package Execute all tests in Section 14, complete the checklist, prepare the report, and create one ZIP containing only the plugin, README, report, and selected screenshots.	README.md; report.pdf; screenshots/; plugin source.	The ZIP installs on a clean local Moodle instance and contains no Moodle core, moodledata, config.php, passwords, session data, credentials, caches, or unrelated personal data.

Tasks must be completed in numerical order unless a dependency has already been satisfied. The final demonstration must show Tasks 1 through 12 on the same local Moodle installation.

5. Problem Description

Each team must build Secure Course Hub, a Moodle local plugin that supports simple course-related help or service requests. The application must provide different behaviors for students and teaching staff while using Moodle's existing accounts, sessions, roles, course enrolments, and security APIs.

A student can create and view only their own requests for a selected course. A teacher with the required capability can view requests associated with an authorized course and update their status. A site administrator can configure or manage the feature through the permissions assigned by Moodle.

Role	Required Operations
Student	View the plugin; create a request for a course in which the student is enrolled; view own requests; update or delete your own request while it is still open.
Teacher / Professor	View requests for a course where access is authorized; filter requests; change request status; add a short response or resolution note.
Administrator / Manager	Access all plugin functions through appropriate Moodle capabilities; assign or verify permissions; inspect configuration and test data.
Guest or unauthenticated user	Must not access protected plugin data or state-changing operations.

Teams may rename the application and improve the domain, but the same authentication, authorization, ownership, server-side, JSON, and security requirements must remain. Any significantly different topic requires instructor or TA approval.

6. Minimum Functional Requirements

- A working local Moodle installation with at least one administrator, one teacher, and two student accounts.

- At least one demonstration course with the teacher and both students enrolled using appropriate roles.
- A local plugin installed under local/securecoursehub or an equivalent approved component name.
- A protected main page that requires an authenticated Moodle user.
- At least two custom capabilities, including one for viewing/creating own requests and one for managing course requests.
- A custom database table storing at least: id, courseid, userid, title, description, status, timecreated, and timemodified.
- Create, read, update, and delete behavior appropriate to the user's permissions and ownership.
- At least one JSON endpoint or Moodle AJAX service called from JavaScript using fetch().
- Dynamic display of records without reloading the entire page for at least one operation.
- Visible success and error feedback.
- CSRF protection for every state-changing request.
- Server-side validation and safe output handling.
- Tests demonstrating authenticated access, denied access, ownership enforcement, and invalid input handling.

7. Required Project Structure

The submitted plugin should use a structure similar to the following. Additional files are permitted when they are clearly organized and documented.

```

local/securecoursehub/
├── version.php
├── index.php
├── ajax.php                # or approved external/AJAX service
├── db/
│   ├── access.php
│   ├── install.xml
│   └── services.php        # only if Moodle external functions are used
├── classes/
│   ├── local/request_service.php
│   └── external/           # optional approved service classes
├── lang/en/local_securecoursehub.php
├── amd/src/dashboard.js    # preferred Moodle JS structure
├── styles.css              # optional
├── README.md
└── pix/                   # optional plugin icon

```

File or Folder	Required Purpose
version.php	Plugin identity, component name, version number, required Moodle version, and maturity information.
db/access.php	Capability definitions and default role archetypes.
db/install.xml	Definition of the custom database table using Moodle XMLDB format.
lang/en/...	Human-readable interface and capability strings; visible text should not be hard-coded repeatedly.
index.php	Secure entry page, Moodle page context, authentication, capability checks, and rendered interface.
classes/local/...	Business and database logic separated from page scripts where practical.
ajax.php or external service	Validated JSON request/response path protected by login, capability, ownership, and CSRF checks.
README.md	Installation, upgrade, test accounts, permissions, run instructions, and known limitations.
report.pdf	Technical report and testing evidence.

7.1 Starter Code Examples

The following starter examples establish the minimum structure. They are starting points only: each team must complete, adapt, test, and explain the code. Copying the examples without implementing the required behaviour does not satisfy the laboratory requirements.

Starter Code 1 - Plugin Metadata (version.php)

```
<?php
// local/securecoursehub/version.php
defined('MOODLE_INTERNAL') || die();

$plugin->component = 'local_securecoursehub';
$plugin->version = 2026071300;
$plugin->requires = 2024100700; // Adjust to the installed Moodle version.
$plugin->maturity = MATURITY_ALPHA;
$plugin->release = '0.1.0';
```

Required completion: verify the installed Moodle version, select an appropriate requires value, increment the plugin version after schema changes, and confirm that Moodle installs the plugin successfully.

Starter Code 2 - Capability Definitions (db/access.php)

```
<?php
// local/securecoursehub/db/access.php
defined('MOODLE_INTERNAL') || die();

$capabilities = [
    'local/securecoursehub:viewown' => [
        'captype' => 'read',
        'contextlevel' => CONTEXT_COURSE,
        'archetypes' => ['student' => CAP_ALLOW, 'editingteacher' => CAP_ALLOW],
    ],
    'local/securecoursehub:managecourserequests' => [
        'captype' => 'write',
        'riskbitmask' => RISK_PERSONAL,
        'contextlevel' => CONTEXT_COURSE,
        'archetypes' => ['editingteacher' => CAP_ALLOW, 'manager' => CAP_ALLOW],
    ],
];
```

Required completion: add the create-request capability, add matching language strings, verify role permissions in the demonstration course, and enforce every capability on the server.

Starter Code 3 - Protected Plugin Page (index.php)

```
<?php
// local/securecoursehub/index.php
require_once(__DIR__ . '/../../config.php');

$courseid = required_param('courseid', PARAM_INT);
$course = get_course($courseid);
require_login($course);

$context = context_course::instance($courseid);
require_capability('local/securecoursehub:viewown', $context);

$PAGE->set_url(new moodle_url('/local/securecoursehub/index.php', ['courseid' => $courseid]));
$PAGE->set_context($context);
$PAGE->set_title(get_string('pluginname', 'local_securecoursehub'));
$PAGE->set_heading(format_string($course->fullname));

echo $OUTPUT->header();
echo $OUTPUT->heading(get_string('pluginname', 'local_securecoursehub'));
echo $OUTPUT->footer();
```

Required completion: render the request form and records, load data through a service class, display output safely, and show only operations authorized for the current user.

Starter Code 4 - Secure JSON Request and Dynamic Update

```
// amd/src/dashboard.js
export async function updateStatus(endpoint, requestId, newStatus) {
  const response = await fetch(endpoint, {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify({
      action: 'update_status',
      id: requestId,
      status: newStatus,
      sesskey: M.cfg.sesskey
    })
  });

  const result = await response.json();
  if (!response.ok || !result.success) {
    throw new Error(result.error || 'The request failed.');
```

```
}
return result;
}

<?php
// Required server-side sequence in ajax.php or an approved external service:
// 1. Load config.php and call require_login().
// 2. Decode and validate the JSON body.
// 3. Validate sesskey before every state-changing operation.
// 4. Establish the course context and check the required capability.
// 5. Check record ownership or authorized course access.
// 6. Use the Moodle Database API to update the record.
// 7. Return a structured JSON success or error response.
```

Required completion: implement the server endpoint, handle expired sessions and denied access, update the page without a full reload, and demonstrate the request and response in browser developer tools.

8. Part A - Required Local Moodle Environment

Install one local Moodle development instance that remains available for the entire laboratory and demonstration. Use an official local package, XAMPP/WAMP/MAMP, Docker, or another approved setup. Do not expose the instance publicly.

- Confirm that the Web server, PHP, database server, and Moodle site start correctly.
- Complete the Moodle installation and create the site administrator account.
- Enable developer-oriented debugging only on the local test site.
- Create one demonstration course and the required teacher/student accounts.
- Enrol users in the course with appropriate Moodle roles.
- Install the team's plugin through the normal Moodle plugin upgrade process.
- Keep the Moodle source, moodledata directory, and database credentials out of the submitted ZIP.
- Document the exact Moodle version and local environment in README.md.

9. Part B - Required Authentication and Session Behaviour

Students must use Moodle's existing authentication and session system. They must not build a separate username/password database, store plaintext passwords, or replace Moodle's login mechanism.

- Call require_login() near the beginning of every protected page or endpoint, using the correct course context where applicable.
- Use the authenticated Moodle user supplied by the platform rather than trusting a userid from the browser.
- Display the current user's name using safe Moodle output functions.
- Demonstrate what happens when an unauthenticated user visits the plugin page.
- Demonstrate logout or expired-session behavior and explain how the request loses its authenticated identity.

- Explain in the report the difference between Moodle's session cookie and the server-side session state.
- Never display, log, transmit, or include the complete session cookie or sesskey in screenshots or the report.

```
require_once(__DIR__ . '/../../config.php');

$courseid = required_param("courseid", PARAM_INT);
$course = get_course($courseid);
require_login($course);

$context = context_course::instance($courseid);
```

10. Part C - Required Authorization and Ownership Rules

Authentication alone is not sufficient. The plugin must use capabilities and resource-level rules to decide what each authenticated user may do.

Capability	Suggested Purpose	Suggested Default Archetypes
local/securecoursehub:viewown	View the plugin and own requests.	Student, teacher, manager
local/securecoursehub:createrequest	Create a request for an enrolled course.	Student, teacher, manager
local/securecoursehub:managecourserquests	View and update requests in an authorized course.	Editing teacher, manager
local/securecoursehub:manageall	Optional site-wide administrative access.	Manager only

- Define capabilities in db/access.php and use a relevant system or course context.
- Use `require_capability()` when access is mandatory and `has_capability()` when selecting an allowed interface path.
- Check course enrolment or course access where required.
- For student operations, verify that the database record `userid` equals the authenticated user id.
- For teacher operations, verify both the required capability and the course associated with the record.
- Do not trust hidden form fields, JavaScript variables, URL parameters, or claimed roles as authorization evidence.
- Return or display a safe access-denied response when a user is authenticated but not permitted.

```
require_capability(
    "local/securecoursehub:viewown",
    $context
);

if ((int)$record->userid !== (int)$USER->id &&
    !has_capability("local/securecoursehub:managecourserquests", $context)) {
    throw new required_capability_exception(
        $context,
        "local/securecoursehub:managecourserquests",
        "nopermissions",
        ""
    );
}
```

11. Part D - Required CRUD and Database Behaviour

The plugin must persist request records using the Moodle Database API. Directly concatenating user input into SQL is not permitted. A starter `install.xml` file may be supplied by the instructor, but every team member must understand the table and operations.

Operation	Required Rule
-----------	---------------

Create	Use the authenticated user id; validate course access, title, description, and permitted initial status; record timestamps.
Read own	Return only records belonging to the current user unless a management capability applies.
Read course	Require the course-management capability in the record's course context.
Update	Students may update only their own open records; teachers may update status/response for authorized course records.
Delete	Allow only under a documented rule, such as the owner deleting an open request or a manager deleting an authorized record.
Not found	Return a clear 404-style result or Moodle exception without revealing unrelated records.
Invalid input	Return a clear validation error and do not write partial or unsafe data.

- Use `required_param()`, `optional_param()`, or validated JSON fields with appropriate `PARAM_*` types.
- Use `$DB` methods and Moodle records rather than raw unparameterized queries.
- Keep database operations in a service class where practical.
- Use `time()` for `timecreated` and `timemodified`.
- Do not expose database errors, stack traces, password hashes, configuration values, or server paths to users.

12. Part E - Required JSON and Dynamic Interface Behaviour

Implement at least one state-changing operation with JavaScript `fetch()` and a JSON response. The operation must update the visible page without a full reload. A teacher status update is the recommended operation.

- Use a clear endpoint or an approved Moodle AJAX/external service.
- Accept and return JSON for the selected operation.
- Include credentials automatically only for the same local Moodle origin; do not copy authentication secrets into JavaScript.
- Validate authentication, capability, context, ownership, and CSRF protection on the server before processing.
- Return structured success and error responses.
- Handle network failure, invalid JSON, unauthenticated/expired sessions, forbidden operations, missing records, and validation errors.
- Update at least one part of the Web page dynamically without a full page reload.
- Keep interface restrictions for usability, but repeat every security rule on the server.

```
const response = await fetch(endpoint, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    action: "update_status",
    id: requestId,
    status: newStatus,
    sesskey: M.cfg.sesskey
  })
});

const result = await response.json();
if (!response.ok) {
  showError(result.error || "The request failed.");
}
```


13. Part F - Mandatory Security and Privacy Controls

Security and privacy are implementation requirements, not report-only discussion topics. The project must demonstrate the following controls.

Control	Required Evidence
Authentication	Protected pages require Moodle login and use the platform-authenticated user.
Authorization	Capabilities are checked server-side in the correct context.
Ownership	Students cannot access or modify another student's record by changing an id.
CSRF protection	Every state-changing request carries a Moodle sesskey and the server validates it.
XSS prevention	Untrusted values are escaped or rendered using appropriate Moodle output functions; innerHTML is avoided for untrusted text.
Input validation	Type, presence, length, allowed values, and business rules are checked on the server.
Injection prevention	Moodle Database API or parameterized queries are used; user input is never concatenated into SQL.
Safe errors	Users receive useful but non-sensitive errors; internal details are not exposed.
Session safety	No session identifiers are printed, stored in project files, or copied into screenshots.
Privacy	Only data required for the feature is collected; access and retention are explained in the report.
Secrets and configuration	No real passwords, tokens, database credentials, config.php, or moodledata files are submitted.
Logging	Security-relevant failures may be logged locally without logging passwords, cookies, sesskeys, or sensitive text.

The report must map at least four implemented controls to relevant risks such as broken access control, identification/authentication failures, injection, security misconfiguration, XSS, CSRF, or logging/monitoring failures.

14. Part G - Mandatory Acceptance Tests

Execute every test listed in the table below using the named account role. Record the expected result, actual result, and Pass/Fail outcome. A submission with an unexecuted mandatory test is incomplete.

Test	Expected Result
Unauthenticated user opens plugin page	Moodle requires login or denies access before plugin data is displayed.
Student creates a valid request	Record is created for the authenticated student and correct course.
Student submits missing/invalid fields	Server rejects the request with a safe validation message.
Student views own records	Only the authenticated student's records are returned.
Student changes another student's record id	Request is denied; no unrelated data is exposed or modified.
Student calls teacher-only operation directly	Server denies the operation even if the UI button is hidden.
Teacher views authorized course records	Authorized records are displayed.
Teacher attempts another unauthorized course	Access is denied according to the capability/context policy.
Missing or invalid sesskey on state change	Operation is rejected and data remains unchanged.
Injected HTML/JavaScript text	Text is displayed safely and does not execute.
Missing record	A clear not-found result is produced without revealing unrelated data.
Expired/logout session during AJAX call	Client shows a useful authentication/session error and does not continue.
Browser and server diagnostics	No unexplained console, PHP, or database errors remain.

Testing evidence must include screenshots or a concise test table showing the account role, request/action, expected result, actual result, and pass/fail outcome. Do not include visible passwords, session cookies, sesskeys, database credentials, or private student information.

15. Moodle Integration and Security Checklist

Item	Completed?
The local Moodle site runs correctly and the version is documented.	
The plugin installs through Moodle without modifying core files.	
The demonstration course and required test accounts are configured.	
Every protected page or endpoint verifies login.	
Capabilities are defined in db/access.php.	
Authorization checks use the correct system or course context.	
Student ownership is checked for record-level operations.	
Teacher access is limited to authorized course records.	
The plugin uses Moodle's Database API.	
Input is validated on the server.	
State-changing requests validate sesskey/CSRF protection.	
Output escaped and injected scripts do not execute.	
At least one JSON/fetch interaction works dynamically.	
Client-side restrictions are duplicated by server-side checks.	
Unauthenticated and forbidden cases are tested.	
Invalid input and missing records are tested.	
No passwords, cookies, sesskeys, config.php, or moodledata content are submitted.	
The README explains installation, permissions, execution, and testing.	
The report explains architecture, AuthN, AuthZ, sessions, security, privacy, and testing.	

16. Laboratory Demonstration Requirement

Each team must demonstrate the laboratory work to the TA. The report will be considered for grading only if the demonstration has been completed. All team members must be present and each student must be able to explain the submission.

- Start the local Moodle environment and open the demonstration course.
- Show the plugin folder structure and explain the purpose of the main files.
- Log in as a student and create/view an own request.
- Attempt an unauthorized student operation and explain why the server denies it.
- Log in as a teacher and perform an authorized course-management operation.
- Show the capability and ownership checks in the code.
- Show the JSON/fetch interaction in browser developer tools.
- Demonstrate CSRF protection by explaining or testing the sesskey requirement safely.

- Show input validation and safe handling of an XSS-style test string.
- Explain the difference among authentication, authorization, role/capability checks, and ownership checks.
- Explain one privacy decision and one known limitation.

17. Deliverables

Each team must submit one ZIP file on Brightspace containing the following items:

- The complete local plugin folder only, not the full Moodle installation.
- All PHP, XML, language, JavaScript, CSS, and image files required by the plugin.
- A README.md file with installation and test instructions.
- A report.pdf file satisfying Section 18.
- A screenshots/ folder containing selected installation, functionality, security, JSON, and testing evidence.
- An optional database export containing only synthetic plugin test data, if explicitly requested by the TA.
- Any instructor-provided starter files that were modified, with the modifications clearly identified.

The ZIP must not include Moodle core, moodledata, config.php, database credentials, real passwords, browser profiles, session cookies, sesskeys, caches, logs containing secrets, or personal information unrelated to the test scenario.

18. Report Requirements

- Course code, laboratory number, team members, and student numbers.
- Short description of Secure Course Hub and the implemented user stories.
- Local environment and exact Moodle version.
- Architecture diagram showing browser, Moodle/PHP, authentication/session layer, authorization/capabilities, plugin logic, and database.
- Plugin file structure and purpose of major files.
- Data model and description of the custom table.
- Authentication and session workflow.
- Roles, capabilities, contexts, and an access-control matrix.
- Ownership/resource-level authorization rules.
- Description of the JSON/fetch interaction with example request and response.
- Input validation, error handling, XSS protection, CSRF protection, and database safety.
- Privacy analysis: data collected, who can access it, retention assumptions, and data minimization.
- Test table covering successful and denied cases.
- Selected screenshots with captions and no exposed secrets.
- Completed checklist.
- Short reflection on problems encountered, solutions, and limitations.
- References for Moodle documentation and any external code or resources used.

19. Grading Rubric

Component	Points
Local Moodle deployment, course configuration, accounts, and repeatable setup	8
Correct plugin structure, installation, language strings, README, and maintainability	10
Authentication and session integration using Moodle mechanisms	10

Capabilities, RBAC, contexts, and server-side authorization	15
Ownership and course-level resource access controls	10
Server-side processing, Database API, CRUD rules, and data integrity	12
JSON/fetch integration and dynamic client-side behavior	10
Security controls: CSRF, XSS, validation, safe errors, injection prevention, and secrets handling	12
Testing evidence, negative tests, debugging, and demonstrated results	6
Laboratory demonstration with all team members present	3
Report quality, privacy analysis, architecture, checklist, and reflection	4
Total	100

20. Important Notes

- **This is a programming laboratory. A Moodle installation without a functioning custom plugin and original code does not satisfy the implementation requirements.**
- Late submissions will not be accepted unless official accommodation or approved exceptional circumstance applies.
- All team members must understand the plugin code, security checks, database operations, JSON exchange, and report.
- Students must not modify Moodle core files. All project functionalities must be implemented as a plugin or through an explicitly approved extension point.
- Students must not implement a separate password store or collect real user passwords.
- Use synthetic test accounts and synthetic request content only.
- External code, templates, icons, images, libraries, or tutorials must be cited and permitted for academic use.
- AI-assisted code must be understood, tested, adapted, and disclosed according to course and University academic integrity requirements.
- A starter plugin skeleton may be provided. Using it does not remove the requirement to understand and explain every submitted part.
- Developer debugging is for the local environment only and must not expose sensitive details in submitted screenshots.
- Teams should create a backup before changing plugin database definitions or upgrading the plugin version.
- Public deployment is not required and should not be attempted unless explicitly approved and properly secured.

21. Suggested Timeline

Date	Suggested Work
Before Monday, July 13	Install the local prerequisites, review the laboratory requirements, and verify that Moodle, PHP, and the database server start correctly.
Monday, July 13	Read the lab statement, form teams, confirm the local Moodle installation, create the four required test accounts and demonstration course, and begin the plugin skeleton.
July 15	Create the plugin skeleton, version metadata, language strings, page setup, and initial installation.
July 16	Define capabilities and configure role permissions in the test course.
July 17	Create the database table and implement record creation and own-record listing.
July 18	Implement teacher course-management behavior and ownership/context checks.

July 19	Add update/delete rules, validation, safe errors, and not-found handling.
July 20	Implement the JSON endpoint or Moodle AJAX service and fetch()-based dynamic behavior.
July 21	Add sesskey/CSRF validation, XSS-safe output, privacy controls, and secrets review.
July 22	Execute functional tests and negative authorization/ownership tests.
July 23	Complete screenshots, test table, architecture diagram, access matrix, and report draft.
Friday, July 24	Complete the clean installation test, final demonstration preparation, debugging, screenshots, report, ZIP review, and secret scan. Submit the ZIP file on Brightspace by 11:59 PM.

22. Recommended Technical References

- Moodle Developer Documentation: Local plugins and common plugin files.
- Moodle Developer Documentation: Access API, roles, capabilities, contexts, require_login(), and require_capability().
- Moodle Security Guidelines: authentication, unauthorized access, cross-site request forgery, cross-site scripting, and input/output handling.
- Moodle Developer Documentation: Database API and XMLDB editor/schema definitions.
- Moodle Developer Documentation: External services and AJAX, when used.
- CSI 3140 Chapter 15: Authenticating and Authorizing Requests.
- CSI 3140 Laboratory 4: Client-Server Interaction and Modern Web Services.